Exact SO(3)-Equivariant Isotropic Kernels for Rotation-Robust Neural Dynamics

Ridham Patel RIDHAM.PATEL@IITGN.AC.IN
Indian Institute of Technology Gandhinagar, Gandhinagar, India

Abstract

Neural surrogates for vector-valued partial differential equations can fit training data yet change their predictions when the same physical state is expressed in a rotated coordinate frame. We study this failure on three-dimensional Navier–Stokes dynamics observed at irregularly placed points. We introduce the Invariant-Conditioned Isotropic Kernel Neural Operator (IKNO), a compact graph model that builds local interactions from scalar quantities unchanged by rotation and vector directions that rotate with the data. Consequently, rotating the positions and velocities rotates the predicted velocity change in exactly the same way. On a held-out test set fixed after model design, training unconstrained graph models on randomly rotated examples reduces but does not eliminate their coordinate dependence. In contrast, IKNO is consistent to numerical precision, matches the forecasting accuracy of a general rotation-aware Tensor Field Network with 5.6 times fewer parameters, and outperforms a parameter-matched graph simulator. These results show that a compact, PDE-specialized model can remove coordinate dependence without sacrificing forecasting accuracy.

Keywords: equivariant neural networks, neural PDE solvers, irregular meshes, Navier–Stokes, geometric deep learning

1. Introduction

The same physical state can be expressed in many coordinate frames. A quantity is invariant if a transformation leaves it unchanged; a map is equivariant if its output transforms with its input. For a three-dimensional velocity field, rotation equivariance requires a joint rotation of coordinates and velocities to rotate the predicted velocity change by the same amount. The vector prediction changes, but the physical rule does not depend on the axes. We study proper three-dimensional rotations, denoted SO(3), which exclude reflections. Without this constraint, low error in the training frame does not determine behavior in a new frame (Villar et al., 2021; Helwig et al., 2023).

This issue is especially relevant for learned partial differential equation (PDE) solvers on irregular meshes. Graph models are usually permutation-equivariant: reordering sampled points merely reorders their outputs. This lets graph simulators accommodate changing discretizations and supports accurate long-horizon prediction (Sanchez-Gonzalez et al., 2020; Pfaff et al., 2021). However, permutation equivariance concerns point labels, not spatial orientation, and does not specify how a vector field transforms under rotation. Treating Cartesian components as ordinary feature channels can therefore make the learned dynamics depend on the axes (Bronstein et al., 2021).

Rotation-equivariant architectures remove this dependence by prescribing how features transform. Tensor-field and steerable networks propagate geometric quantities according to

© 2026 Ridham Patel.

their rotation rules (Thomas et al., 2018; Fuchs et al., 2020), while lighter-weight constructions use vector-valued features or invariant scalar weights (Satorras et al., 2021; Villar et al., 2021). We ask whether such an exact constraint improves forecasting under a coordinate-frame shift without reducing accuracy in the observed frame. We compare it with rotation augmentation, which trains a model on randomly rotated copies of the observed states (Brandstetter et al., 2022b; Puny et al., 2022).

We use a controlled one-frame prediction experiment on three-dimensional incompressible Navier–Stokes flow. The model maps a velocity field sampled at irregular points to a one-step derivative estimate. Training and validation use orientation-biased initial conditions; testing uses both the original states and jointly rotated coordinates, velocities, and targets. Because the rotated target is known exactly by changing coordinates, additional disagreement isolates frame dependence without a second simulation.

To perform this test, we introduce the Invariant-Conditioned Isotropic Kernel Neural Operator (IKNO). Each edge coefficient is produced from scalar rotation invariants. The resulting coefficients weight four vector bases that rotate with the data. The construction has three properties relevant to the experiment: it is exactly SO(3)-equivariant, it annihilates constant fields in its difference channels, and it contains only 4,869 trainable parameters.

The main result is that architectural equivariance and rotation augmentation produce measurably different operators. Augmentation reduces the rotation sensitivity of two graph controls but does not eliminate it; IKNO remains unchanged across 1,000 sampled coordinate frames. A Tensor Field Network provides the complementary exact-equivariance control: it matches IKNO in forecasting accuracy but uses 5.6 times more parameters. A parameter-matched graph simulator is substantially less accurate. The evidence therefore supports a specific conclusion: the specialized isotropic construction provides exact rotation robustness at low parameter cost.

2. Related Work

Equivariant representation learning formalizes weight sharing over transformations (Cohen and Welling, 2016; Bronstein et al., 2021). Tensor Field Networks and SE(3)-Transformers use typed features that follow irreducible group representations and spherical harmonics, a basis for angular dependence (Thomas et al., 2018; Fuchs et al., 2020); e3nn implements their tensor products (Geiger and Smidt, 2022). Lighter Cartesian constructions include EGNN, Vector Neurons, PaiNN, and SEGNN (Satorras et al., 2021; Deng et al., 2021; Schütt et al., 2021; Brandstetter et al., 2022a). Classical isotropic representation theory and its modern scalar formulation express equivariant vector functions through invariant coefficients and equivariant generators (Smith, 1971; Villar et al., 2021). The latter also distinguishes proper rotations from reflections through cross-product generators. IKNO selects a compact, PDE-oriented subset of admissible generators on an irregular mesh; it does not claim a complete parameterization.

Learned PDE models take several complementary architectural forms. Neural operators learn maps between function spaces and can amortize repeated PDE solution (Li et al., 2021). Graph simulators instead represent local physical interactions by message passing, which naturally supports particles and irregular meshes (Sanchez-Gonzalez et al., 2020; Pfaff et al., 2021). Message-passing PDE solvers connect these learned updates to classical

local discretizations (Brandstetter et al., 2022c). Symmetry has been incorporated through data augmentation derived from Lie point symmetries (Brandstetter et al., 2022b), steerable differential operators (Jenner and Weiler, 2022), and group equivariant Fourier layers (Helwig et al., 2023). LieConv treats arbitrary continuous data (Finzi et al., 2020), while Neural ePDOs learn adaptive equivariant differential operators (He et al., 2023). In contrast to Fourier methods on regular grids, IKNO acts directly on irregular three-dimensional samples. Our experiment also measures operator equivariance separately from forecasting error, which distinguishes exact architectural consistency from good performance on one orientation distribution.

3. Problem Setting

We first define the evolution problem and the symmetry to be tested. Let $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^3$ be sample coordinates and let $\boldsymbol{u}_i(t) \in \mathbb{R}^3$ denote velocity. A learned local operator f_θ predicts the derivative

$$\dot{\boldsymbol{u}}_i(t) = f_{\theta,i}(\boldsymbol{U}(t), \boldsymbol{X}), \qquad \boldsymbol{U} = (\boldsymbol{u}_1, \ldots, \boldsymbol{u}_N). \tag{1}$$

We integrate this derivative with an explicit Euler step of duration Δt to obtain autoregressive forecasts.

The relevant transformation rotates coordinates and vector components together. For $R \in \mathrm{SO}(3)$, define $R\boldsymbol{X} = \{R\boldsymbol{x}_i\}_i$ and $R\boldsymbol{U} = \{R\boldsymbol{u}_i\}_i$. The operator is SO(3)-equivariant when

$$f_\theta(R\boldsymbol{U}, R\boldsymbol{X}) = Rf_\theta(\boldsymbol{U}, \boldsymbol{X}) \quad \text{for every } R \in \mathrm{SO}(3). \tag{2}$$

Equation 2 is stronger than low rotated prediction error. The equation is an operator identity; rotated prediction error also depends on the difficulty of the forecasting task. We therefore report both quantities.

Our physical system is the incompressible Navier–Stokes equation

$$\partial_t \boldsymbol{u} + (\boldsymbol{u} \cdot \nabla)\boldsymbol{u} = -\nabla p + \nu \nabla^2 \boldsymbol{u}, \qquad \nabla \cdot \boldsymbol{u} = 0, \tag{3}$$

on a periodic three-dimensional domain. The differential terms in Equation 3 are coordinate-covariant. Consequently, jointly rotating coordinates and vector values gives a valid representation of the same local dynamics, even when the observed initial-condition distribution favors one orientation (Olver, 1993; Brandstetter et al., 2022b).

4. Invariant-Conditioned Isotropic Kernel

The method must use local directional information without tying a prediction to a global axis. Motivated by isotropic representation results, we separate each message into invariant coefficients and equivariant vector generators (Smith, 1971; Villar et al., 2021).

4.1. Invariant edge description

For a directed edge from node i to node j, define the relative coordinate, unit direction, and velocity difference

$$\boldsymbol{r}_{ij} = \boldsymbol{x}_j - \boldsymbol{x}_i, \qquad \widehat{\boldsymbol{r}}_{ij} = \frac{\boldsymbol{r}_{ij}}{\|\boldsymbol{r}_{ij}\|_2}, \qquad \delta\boldsymbol{u}_{ij} = \boldsymbol{u}_j - \boldsymbol{u}_i. \tag{4}$$

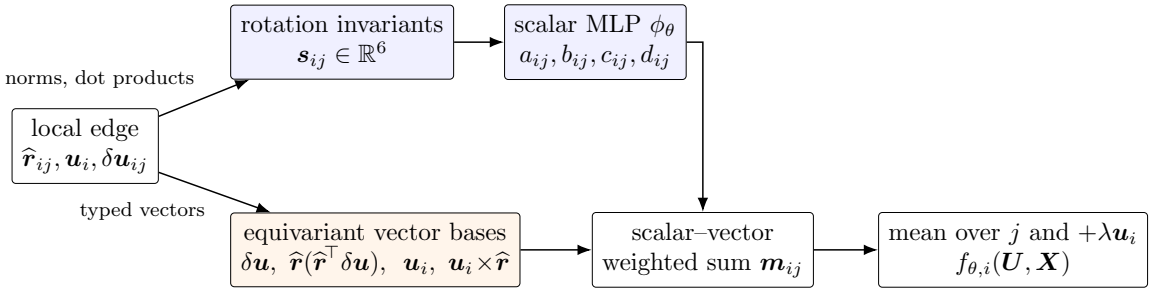


Figure 1: IKNO message construction. Invariant coefficients weight equivariant vector bases; aggregation preserves their common rotation and therefore yields an equivariant output.

The edge graph connects points within radius $r_{\max}$ and retains at most K neighbors. Because this rule depends only on distance, a rotation does not change the graph except for floating-point tie cases.

The coefficient network receives six scalar invariants:

$$s_{ij} = \left[\|r_{ij}\|_2, \|u_i\|_2, \|\delta u_{ij}\|_2, u_i^\top \widehat{r}_{ij}, \delta u_{ij}^\top \widehat{r}_{ij}, u_i^\top \delta u_{ij}\right]. \tag{5}$$

Apart from radial scale $\|r_{ij}\|_2$ and the fixed norm $\|\widehat{r}_{ij}\|_2 = 1$, these quantities encode the nonconstant Gram information of $(\widehat{r}_{ij}, u_i, \delta u_{ij})$. Inner products generate rotation-invariant scalar descriptions of vector inputs (Villar et al., 2021). A multilayer perceptron $\phi_\theta : \mathbb{R}^6 \to \mathbb{R}^4$ maps them to $(a_{ij}, b_{ij}, c_{ij}, d_{ij})$.

4.2. Equivariant message and update

Guided by the same representation results, we choose a compact PDE-oriented subset of admissible SO(3)-equivariant generators rather than a universal basis (Smith, 1971; Villar et al., 2021):

$$m_{ij} = a_{ij}\delta u_{ij} + b_{ij}\widehat{r}_{ij}(\widehat{r}_{ij}^\top \delta u_{ij}) + c_{ij}u_i + d_{ij}(u_i \times \widehat{r}_{ij}). \tag{6}$$

The first two terms span direct and longitudinal difference responses. The third permits state-dependent local dynamics. The cross product is orientation-sensitive but rotation-consistent; as a pseudovector under reflections, it distinguishes SO(3) from O(3) (Villar et al., 2021).

Messages are mean-aggregated and combined with a scalar self-interaction:

$$f_{\theta,i}(U, X) = \lambda u_i + \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} m_{ij}. \tag{7}$$

The difference channels vanish on constant fields, matching the nullspace of local differential responses (Brandstetter et al., 2022c; Jenner and Weiler, 2022). The coefficient network has two hidden layers of 64 SiLU units. For optimization stability, its final layer and λ are initialized to zero, so training begins from a stationary derivative.

Proposition 1 *If $\mathcal{N}(i)$ depends only on pairwise distances, then for every $R \in \mathrm{SO}(3)$ and $\boldsymbol{a} \in \mathbb{R}^3$, $f_\theta(R\boldsymbol{U}, R\boldsymbol{X} + \boldsymbol{a}) = Rf_\theta(\boldsymbol{U}, \boldsymbol{X})$.*

Proof Under a joint rotation, each entry of $\boldsymbol{s}_{ij}$ is unchanged, so the four scalar coefficients are unchanged. Each vector basis in Equation 6 is multiplied by R: this follows from preservation of inner products and from $(R\boldsymbol{a}) \times (R\boldsymbol{b}) = R(\boldsymbol{a} \times \boldsymbol{b})$ for $R \in \mathrm{SO}(3)$. Distance-based neighborhoods are unchanged, and summation and scalar self-interaction commute with R. Therefore each output is multiplied by R. A global translation cancels in every $\boldsymbol{r}_{ij} = \boldsymbol{x}_j - \boldsymbol{x}_i$, proving the translation claim. ■

5. Experimental Design

The experiments test one question: does exact operator equivariance improve prediction under an unseen coordinate frame while preserving useful accuracy in the observed frame? We first describe the data and controls, then define metrics that distinguish forecasting accuracy from operator consistency.

5.1. Data and split protocol

We generate three-dimensional incompressible flow on $[0, 2\pi]^3$ using a 16^3 pseudo-spectral solver with two-thirds dealiasing, Leray projection, and Strang-split diffusion (Canuto et al., 1988; Strang, 1968). A fixed-axis large-scale forcing is used for 50 warm-up steps to create an orientation-biased initial state. Each trajectory adds an independent 10% divergence-free perturbation, chosen to diversify trajectories while retaining the warm-up orientation bias, and is then evolved without forcing or observation noise. This separation is deliberate: the forecast dynamics obey rotational covariance, while the observed initial-condition distribution supplies only a restricted set of orientations.

Each trajectory contains 20 snapshots separated by $\Delta t = 0.02$. The spectral velocity is trilinearly interpolated to a separately sampled irregular cloud of 500 points in normalized coordinates. Training uses random 128-point subsamples. The supervised derivative target is $(\boldsymbol{U}(t + \Delta t) - \boldsymbol{U}(t))/\Delta t$. Evaluation uses a fixed 128-point subsample per trajectory.

The development data contain 25 trajectories with seeds 342–366. The first 20 form the model-development pool: a deterministic split assigns 16 to optimization and four to validation, and validation error alone selects checkpoints. Five additional trajectories with seeds 362–366 screen which new controls warrant final evaluation; no screening result appears in the paper. The proposed architecture was frozen before generating the final test set. The added controls were specified and screened only on development data, then fixed before their single final evaluation. The reported test set contains five trajectories with seeds 367–371 and new point clouds.

5.2. Controls and optimization

We compare with persistence and three learned architecture families. Persistence copies the initial field and establishes whether a learned derivative improves over zero dynamics. Graph Network-based Simulator (GNS) uses six message-passing layers following Sanchez-Gonzalez et al. (2020). The second control adapts the EGNN implementation of Satorras et al. (2021)

by passing the three velocity components as node-feature channels; we call it component-EGNN. This is an unconstrained vector-field control, not an equivariant implementation of typed vector features: the original EGNN guarantee applies to coordinate updates with invariant node features.

To distinguish the proposed specialization from general equivariant message passing, we implement a Tensor Field Network (TFN) (Thomas et al., 2018) with e3nn (Geiger and Smidt, 2022). It carries even scalars and odd vectors, and applies distance-conditioned tensor products with spherical harmonics through degree one. The TFN is O(3)-equivariant, and therefore also SO(3)-equivariant. Unlike IKNO, it propagates learned scalar and vector hidden channels through tensor products.

We introduce two controls for possible confounds. First, GNS and component-EGNN are retrained with a fresh Haar-random joint rotation of coordinates, velocities, and derivative targets for every training sample. We sample these rotations by Gaussian QR factorization (Mezzadri, 2007). These variants test approximate symmetry learned through augmentation (Brandstetter et al., 2022b; Puny et al., 2022). Second, a six-layer GNS with hidden width nine contains 5,352 parameters and controls for the capacity difference between the original GNS and IKNO.

All learned methods use the same radius $r_{\max} = 0.4$, at most 64 neighbors, the same trajectory split, and the same temporal-window sampling protocol. IKNO uses 4,869 parameters, TFN 27,297, component-EGNN 30,058, and GNS 63,174. We train three seeds for 100 epochs with Adam and cosine learning-rate decay (Kingma and Ba, 2015), using validation-selected checkpoints. The initial learning rate is 3×10^{-3} for IKNO and 2×10^{-3} for the controls. Additional implementation details and the two-frame extrapolation control are reported in Appendix B.

5.3. Metrics and rotation intervention

Forecasting error at horizon h is the relative Frobenius error

$$e_h = \frac{\|\widehat{\boldsymbol{U}}_h - \boldsymbol{U}_h\|_F}{\|\boldsymbol{U}_h\|_F}. \tag{8}$$

We report $h = 1$ and $h = 10$. To measure coordinate-frame generalization, we draw 1,000 fixed Haar-random rotations for one-step prediction and use the first 20 for ten-step rollouts, concentrating the denser stress test on the cheaper one-step evaluation. The same rotations are applied to every method, coordinate set, input velocity, and target velocity. The resulting rotated error tests the same physical state in an unseen frame.

Average rotated error can conceal a sensitive coordinate frame. We therefore also measure one-step degradation relative to the original frame. Here $e_1(\boldsymbol{X}, \boldsymbol{U})$ denotes the one-step error evaluated on the specified coordinates and input velocity:

$$\Delta_1(R) = e_1(R\boldsymbol{X}, R\boldsymbol{U}) - e_1(\boldsymbol{X}, \boldsymbol{U}), \tag{9}$$

and report its mean, 95th percentile, and sampled maximum over the 1,000 rotations. The maximum is a finite-sample stress test, not a certified worst-case bound.

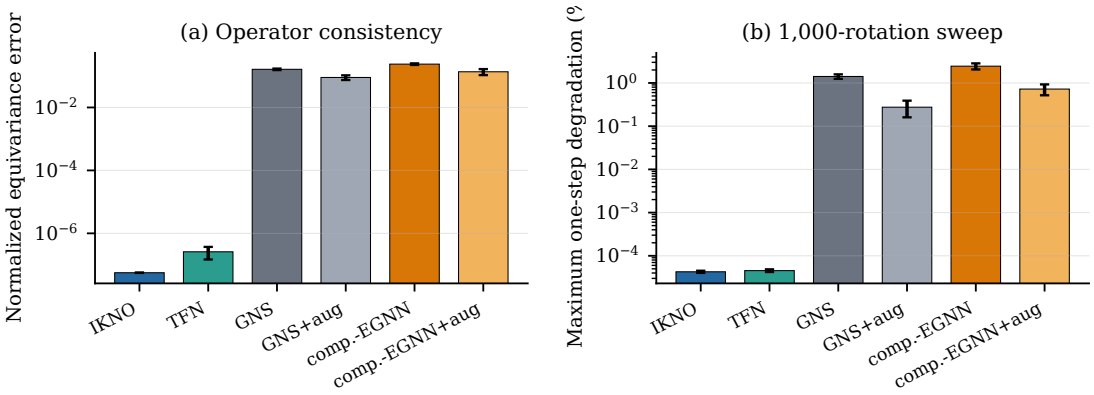


Figure 2: Architectural equivariance differs from rotation augmentation. Bars and error bars show mean and sample standard deviation over three training seeds on the post-design holdout. (a) Augmentation reduces but does not eliminate operator inconsistency; IKNO and TFN are equivariant to numerical precision. (b) The sampled maximum of $\Delta_1(R)/e_1$ over 1,000 rotations remains nonzero after augmentation, whereas both exact architectures are stable up to numerical precision.

Forecasting error can remain small even when the learned derivative is not equivariant. We therefore define our normalized operator equivariance diagnostic

$$e_{\text{eq}} = \mathbb{E}_R \left[\frac{\text{RMS}(f_\theta(R\boldsymbol{U}, R\boldsymbol{X}) - Rf_\theta(\boldsymbol{U}, \boldsymbol{X}))}{\text{RMS}(f_\theta(R\boldsymbol{U}, R\boldsymbol{X})) + \text{RMS}(Rf_\theta(\boldsymbol{U}, \boldsymbol{X}))} \right]. \tag{10}$$

A value near zero means that the learned derivative is coordinate-consistent: rotating the input rotates the derivative by the same amount. We average each test metric over five trajectories within a run and report mean ± sample standard deviation over three training seeds.

6. Results

6.1. Main result: architecture and augmentation produce different symmetry

The main experiment asks whether rotation augmentation recovers the same operator-level consistency as an equivariant architecture. It does not. Figure 2a shows normalized equivariance errors of 0.163 for GNS and 0.238 for component-EGNN. Augmentation reduces these errors by 45% and 43%, respectively, but leaves residual errors of 0.090 and 0.136. In contrast, IKNO reaches 5.60×10^{-8} and TFN reaches 2.59×10^{-7}. These values match the numerical precision observed for the theoretically equivariant implementations (Appendix B).

The 1,000-rotation sweep establishes that the residual operator error affects prediction in particular frames. Figure 2b reports the sampled maximum degradation as a percentage of original-frame error. The value is 1.41% for GNS and 2.45% for component-EGNN. Augmentation reduces these values to 0.27% and 0.72%, but does not eliminate them.

Table 1: Post-design holdout means over three training seeds and five test trajectories (lower is better); rotated e_1/e_{10} use 1,000/20 fixed rotations. The lower panel reports IKNO relative to each baseline: positive reductions and ratios above one favor IKNO. Underlining marks its wins over both augmented controls and, after rotation, every unconstrained control; the single bold entry is the lowest e_{eq}. Persistence has undefined derivative-normalized equivariance error. Per-seed results are in Appendix A.

Method	Params	e_1	e_{10}	Rot. e_1	Rot. e_{10}	e_{eq}
Persistence	0	.012219	.120160	.012219	.120160	—
GNS	63,174	.011451	.111186	.011533	.112025	.163
GNS+aug.	63,174	.011516	.111798	.011507	.111706	.090
comp.-EGNN	30,058	.011418	.110843	.011584	.112529	.238
comp.-EGNN+aug.	30,058	.011505	.111677	.011526	.111905	.136
TFN	27,297	.011489	.111487	.011489	.111487	2.59×10^{-7}
IKNO (ours)	4,869	.011493	.111488	.011493	.111488	$\mathbf{5.60\times10^{-8}}$

Baseline	Δe_1	Δe_{10}	Δ Rot. e_1	Δ Rot. e_{10}	Param. ratio	Eq. ratio	Max-deg. ratio
GNS	−0.36%	−0.27%	+0.35%	+0.48%	12.97×	2.91×10^{6}	3.31×10^{4}
GNS+aug.	+0.20%	+0.28%	+0.12%	+0.19%	12.97×	1.60×10^{6}	6.44×10^{3}
comp.-EGNN	−0.65%	−0.58%	+0.79%	+0.92%	6.17×	4.25×10^{6}	5.73×10^{4}
comp.-EGNN+aug.	+0.11%	+0.17%	+0.29%	+0.37%	6.17×	2.43×10^{6}	1.69×10^{4}
TFN	−0.03%	−0.001%	−0.03%	−0.001%	5.61×	4.63×	1.06×
GNS-small	+3.22%	+4.03%	+3.45%	+4.30%	1.10×	9.27×10^{6}	2.50×10^{4}

The corresponding values are $4.3 \times 10^{-5}\%$ for IKNO and $4.5 \times 10^{-5}\%$ for TFN. Thus the distinction between approximate and exact operator consistency is visible even when their average forecasting errors are close.

The lower panel makes the pairwise forecasting comparisons explicit rather than retaining only global column winners. Component-EGNN is more accurate in the original orientation, but IKNO lowers its rotated errors at both horizons. Against its augmented counterpart, IKNO lowers all four forecasting errors. These descriptive differences accompany reductions by several orders of magnitude in operator inconsistency and sampled worst-frame degradation.

Table 1 answers a second question: does the specialized operator sacrifice accuracy relative to a general equivariant network? TFN and IKNO differ by only 0.03% at one step and less than 0.001% at ten steps. Exploratory two-sided paired t-tests give $p = 0.706$ and $p = 0.994$, respectively. Their forecasting performance is therefore indistinguishable in this experiment, while TFN uses 5.6 times more parameters and spherical harmonics. This comparison supports an efficiency claim, not an accuracy advantage over TFN.

6.2. Supporting result: controlling for parameter count

The preceding comparison leaves open whether the large unconstrained controls are disadvantaged because capacity permits orientation-specific fitting. We isolate capacity with a six-layer, 5,352-parameter GNS. This parameter-matched model obtains rotated errors of 0.011903 at one step and 0.116500 at ten steps. IKNO reduces these errors by 3.45% and

4.30%, respectively; exploratory paired tests across seeds give $p = 0.014$ at both horizons. The corresponding original-frame reductions are 3.22% and 4.03%. The advantage therefore persists when parameter count and message-passing depth are controlled.

6.3. Supporting result: learning nontrivial dynamics

Exact equivariance would be uninformative if obtained by predicting no dynamics. Persistence provides this control because it is exactly equivariant but copies the input field. IKNO reduces persistence error by 5.95% at one step and 7.22% at ten steps. Per-seed results in Appendix A further show that the aggregate comparisons are not driven by a single run.

7. Discussion and Limitations

Low in-frame error measures fit to an observed orientation distribution, whereas Equation 10 tests whether the derivative is a coordinate-consistent map. Augmentation improves both quantities but leaves nonzero operator error and worst-frame degradation, consistent with the distinction between sampled augmentation and exact constructions (Brandstetter et al., 2022b; Puny et al., 2022). IKNO and TFN remove that degradation by construction.

The scope of this conclusion is limited. First, the final test contains five trajectories from one low-resolution Navier–Stokes generator. The evidence does not establish superiority across PDEs, Reynolds numbers, geometries, or boundary conditions. Second, the warm-up uses a fixed-axis forcing, whereas forecast trajectories are unforced. This choice creates the orientation shift needed for the controlled test, but it is not an isotropic sampling of initial conditions. Third, arbitrary rotations change the coordinate description of the periodic box; the test is a joint coordinate-frame transformation of sampled states rather than a second solver run on an independently rotated grid.

The baselines also delimit the claim. Component-EGNN is intentionally an unconstrained adaptation for vector fields and should not be interpreted as a failure of typed scalar–vector models such as PaiNN or SEGNN (Schütt et al., 2021; Brandstetter et al., 2022a). The TFN supplies typed scalar and vector features, but is restricted to degree-one spherical harmonics and one chosen width; it does not exhaust the space of steerable architectures. Its matched accuracy supports the parameter-efficiency of the specialized basis on this task, not a general efficiency ordering. The two-frame extrapolator in Appendix B is much more accurate because the trajectories are temporally smooth; the present result is therefore specific to one-frame operator identification, where the derivative must be inferred from spatial structure.

8. Conclusion

IKNO combines invariant scalar edge functions with an equivariant vector basis to guarantee SO(3) equivariance. On an untouched Navier–Stokes holdout, it matches a tensor-field network with 5.6 times fewer parameters, removes coordinate dependence left by augmentation, and beats a parameter-matched control. This supports compact, PDE-specialized exact operator consistency.

References

Johannes Brandstetter, Rob Hesselink, Elise van der Pol, Erik J. Bekkers, and Max Welling. Geometric and physical quantities improve E(3) equivariant message passing. In *International Conference on Learning Representations*, 2022a.

Johannes Brandstetter, Max Welling, and Daniel E. Worrall. Lie point symmetry data augmentation for neural PDE solvers. In *International Conference on Machine Learning*, pages 2241–2256, 2022b.

Johannes Brandstetter, Daniel E. Worrall, and Max Welling. Message passing neural PDE solvers. In *International Conference on Learning Representations*, 2022c.

Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Velickovic. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods in Fluid Dynamics*. Springer, 1988.

Taco Cohen and Max Welling. Group equivariant convolutional networks. In *International Conference on Machine Learning*, pages 2990–2999, 2016.

Congyue Deng, Or Litany, Yueqi Duan, Adrien Poulenard, Andrea Tagliasacchi, and Leonidas J. Guibas. Vector neurons: A general framework for SO(3)-equivariant networks. In *IEEE/CVF International Conference on Computer Vision*, pages 12200–12209, 2021.

Marc Finzi, Samuel Stanton, Pavel Izmailov, and Andrew Gordon Wilson. Generalizing convolutional neural networks for equivariance to lie groups on arbitrary continuous data. In *International Conference on Machine Learning*, pages 3165–3176, 2020.

Fabian Fuchs, Daniel Worrall, Volker Fischer, and Max Welling. SE(3)-transformers: 3d roto-translation equivariant attention networks. In *Advances in Neural Information Processing Systems*, volume 33, pages 1970–1981, 2020.

Mario Geiger and Tess Smidt. e3nn: Euclidean neural networks. *arXiv preprint arXiv:2207.09453*, 2022.

Lingshen He, Yuxuan Chen, Zhengyang Shen, Yibo Yang, and Zhouchen Lin. Neural ePDOs: Spatially adaptive equivariant partial differential operator based networks. In *International Conference on Learning Representations*, 2023.

Jacob Helwig, Xuan Zhang, Cong Fu, Jerry Kurtin, Stephan Wojtowytsch, and Shuiwang Ji. Group equivariant Fourier neural operators for partial differential equations. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 12907–12930. PMLR, 23–29 Jul 2023.

Erik Jenner and Maurice Weiler. Steerable partial differential operators for equivariant neural networks. In *International Conference on Learning Representations*, 2022.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

Francesco Mezzadri. How to generate random matrices from the classical compact groups, 2007.

Peter J. Olver. *Applications of Lie Groups to Differential Equations*. Springer, 2 edition, 1993.

Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021.

Omri Puny, Matan Atzmon, Heli Ben-Hamu, Ishan Misra, Aditya Grover, Edward J. Smith, and Yaron Lipman. Frame averaging for invariant and equivariant network design. In *International Conference on Learning Representations*, 2022.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning*, pages 8459–8468, 2020.

Víctor Garcia Satorras, Emiel Hoogeboom, and Max Welling. E(n) equivariant graph neural networks. In *International Conference on Machine Learning*, pages 9323–9332, 2021.

Kristof Schütt, Oliver Unke, and Michael Gastegger. Equivariant message passing for the prediction of tensorial properties and molecular spectra. In *International Conference on Machine Learning*, pages 9377–9388, 2021.

G. F. Smith. On isotropic functions of symmetric tensors, skew-symmetric tensors and vectors. *International Journal of Engineering Science*, 9:899–916, 1971. doi: 10.1016/0020-7225(71)90023-1.

Gilbert Strang. On the construction and comparison of difference schemes. *SIAM Journal on Numerical Analysis*, 5(3):506–517, 1968.

Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. Tensor field networks: Rotation- and translation-equivariant neural networks for 3d point clouds. *arXiv preprint arXiv:1802.08219*, 2018.

Soledad Villar, David W. Hogg, Kate Storey-Fisher, Weichi Yao, and Ben Blum-Smith. Scalars are universal: Equivariant machine learning, structured like classical physics. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Appendix A. Per-Seed Results

Table 2 reports the values underlying the aggregate main result for the exact architectures, augmented controls, and parameter-matched GNS. The same 1,000 one-step rotations and 20 ten-step rotations are used for every method and seed.

Table 2: Per-seed post-design holdout results.

Method	Seed	e_1	e_{10}	Rot. e_1	Rot. e_{10}	e_{eq}
IKNO	0	.011498	.111562	.011498	.111562	5.65×10^{-8}
IKNO	1	.011491	.111461	.011491	.111461	5.52×10^{-8}
IKNO	2	.011489	.111442	.011489	.111442	5.64×10^{-8}
TFN	0	.011480	.111346	.011480	.111346	2.36×10^{-7}
TFN	1	.011499	.111621	.011499	.111621	3.81×10^{-7}
TFN	2	.011489	.111495	.011489	.111495	1.61×10^{-7}
GNS + aug.	0	.011515	.111767	.011502	.111657	.074
GNS + aug.	1	.011532	.111940	.011510	.111741	.105
GNS + aug.	2	.011502	.111686	.011508	.111719	.091
comp.-EGNN + aug.	0	.011501	.111632	.011541	.112063	.170
comp.-EGNN + aug.	1	.011516	.111801	.011520	.111824	.123
comp.-EGNN + aug.	2	.011498	.111599	.011518	.111829	.116
GNS-small	0	.011904	.116483	.011902	.116470	.511
GNS-small	1	.011763	.114880	.011820	.115504	.476
GNS-small	2	.011958	.117163	.011988	.117524	.571

Appendix B. Implementation and Additional Controls

All models follow the same optimization and selection protocol. Every training sample contributes four randomly selected one-step temporal windows per epoch. Models minimize mean-squared derivative error, clip gradient norms at 10, and use weight decay 10^{-5}. Checkpoint selection minimizes validation one-step relative error. The test split is evaluated once after freezing the architecture and selecting the checkpoint.

Two simple temporal controls characterize the predictability of the dataset. Persistence receives one frame and obtains $e_1 = 0.012219$ and $e_{10} = 0.120160$. A two-frame linear extrapolator estimates the increment from the first two observations and obtains $e_1 = 0.000259$ and $e_{10} = 0.014188$. The latter is not a like-for-like baseline because it receives an additional frame that nearly reveals the time derivative. Its performance shows that this dataset is temporally smooth and that the main experiment specifically tests one-frame spatial operator learning.

Numerical equivariance can differ from the exact algebraic result because the graph is constructed in floating-point arithmetic. The theoretical graph is unchanged by rotation because pairwise distances are preserved. In floating-point arithmetic, points exactly at the radius boundary could change adjacency. No such instability is observed in the reported graphs: IKNO prediction error is unchanged to the displayed precision and the residual operator error is on the order of single-precision rounding.